Belief-Aware RL Agent: Design and Experiments

Belief-Aware RL Project

May 2026

1 Aim

We design and evaluate a belief-aware reinforcement learning agent for the ABIDES daily investor environment. The central hypothesis is that an agent conditioned on an estimate of the latent market regime — the hidden composition of background traders — will outperform a standard agent that sees only public market observations.

Formally, we compare two policy classes:

$$\pi_{\text{base}}(o_t) \quad \text{vs.} \quad \pi_{\text{belief}}(o_t, b_t),$$

where o_t is the standard market observation vector and b_t is a belief vector over market regimes, estimated from recent public history.

2 Agent Design

2.1 Observation Space

The observation o_t follows the standard ABIDES daily investor interface:

Signal	**Meaning**
$o_t[0]$	Holdings (signed share inventory)
$o_t[1]$	Imbalance (bid volume / total visible volume)
$o_t[2]$	Spread (best ask − best bid)
$o_t[3]$	Direction feature (mid price − last transaction price)
$o_t[4{:}7]$	Lagged mid-price returns

Table 1: Standard observation features.

The full observation for the belief-aware agent is the concatenation $[o_t, b_t]$, where $b_t \in \Delta^{K-1}$ is a K-dimensional probability simplex over market regimes (Section 3).

2.2 Action Space

At each timestep the agent selects one of three actions:

$$a_t \in \{0 \text{ (market buy)},\ 1 \text{ (hold)},\ 2 \text{ (market sell)}\}.$$

2.3 Reward Function

At each RL step t the simulator reports the investor's signed share inventory $q_t \in \mathbb{R}$, cash balance $c_t \in \mathbb{R}$, and last traded price P_t (mid or last-transaction convention as in ABIDES). The **mark-to-market wealth** at time t is

$$W_t := c_t + q_t P_t. \tag{1}$$

This is standard accounting cash plus holdings valued at P_t.[1]

Dense incremental reward. In *dense* reward mode the environment returns the one-step gain in wealth:

$$r_t := W_t - W_{t-1}, \tag{2}$$

with W_{-1} or W_0 set from the episode's initial portfolio after reset. Thus r_t is the instantaneous capital change due to executions, dividends (if modeled), holding gains on inventory as P_t moves between updates, and any cash injections—all mediated by ABIDES's microstructure updates between wakes of the RL agent.

Cumulative payoff and telescoping. Over an episode horizon $t = 1, \ldots, T$,

$$\sum_{t=1}^{T} r_t \;=\; \sum_{t=1}^{T} (W_t - W_{t-1}) \;=\; W_T - W_0, \tag{3}$$

because interior terms cancel (*telescoping*). So maximizing expected discounted cumulative reward is alignment with increasing terminal wealth W_T relative to W_0, up to any terminal/sparse bonuses the experiment adds beyond (2).

Implementation. In dense mode the daily investor env computes mark-to-market as cash plus holdings valued at the last transaction price, takes the step-to-step difference $W_t - W_{t-1}$, then divides by the fixed order size and by an approximate number of agent steps per trading day. That keeps rewards on a reasonable scale for PPO; the economic meaning is still the change in W_t.

The belief vector b_t is an *input* to the policy only; it does not appear in (1)–(2). This keeps the reward signal clean and decouples belief quality from the definition of profit.

2.4 Policy Training

Both π_{base} and π_{belief} are trained with PPO in a two-phase pipeline:

1. **Phase 1 (belief pretraining):** generate ABIDES rollouts across the regime sweep, collect $(h_t, \hat{r})$ labels, train the belief classifier, then **freeze** its weights.
2. **Phase 2 (policy training):** train the RL policy with the frozen belief module as a feature extractor only; no joint fine-tuning is performed in this stage.

This enforces a clean separation between regime inference and action selection.

For the current experiment cycle, we first run an oracle version of this pipeline before training a learned belief model. This gives a fast answer to the most important question: whether regime information helps the trading policy at all. If the oracle signal improves PPO, then the learned belief module is a meaningful next target; if it does not, the bottleneck is likely the policy, reward, or market setup rather than belief inference.

[1] Implementation uses the raw last market transaction price in `daily_investor`; the observation feature "direction" compares mid to that price.

3 Belief Module

3.1 Regime Definitions

We define $K = 3$ discrete market regimes:

$$\mathcal{R} = \{\texttt{momentum},\ \texttt{value},\ \texttt{noise}\}.$$

Each regime corresponds to a market composition in which one agent type dominates price dynamics. The belief vector is:

$$b_t = \big[p(\texttt{momentum} \mid h_t),\ p(\texttt{value} \mid h_t),\ p(\texttt{noise} \mid h_t)\big] \in \Delta^2,$$

where h_t is a recent window of public observations $\{o_{t-k}, \ldots, o_t\}$.

3.2 Oracle Belief (Ground Truth)

Since ABIDES is a simulator, the ground truth regime is known at episode start from the background market parameters. The oracle maps these parameters to a hard regime label via a simple heuristic threshold rule:

$$\hat{r} = f(\texttt{num_value_agents},\ \texttt{num_momentum_agents},\ \ldots)$$

The oracle b_t^{oracle} is then the one-hot vector corresponding to $\hat{r}$.

Primary sweep parameters for regime definition:

Parameter	Regime signal
`num_value_agents`	Strongest signal; high count → `value` regime
`num_momentum_agents`	High count → `momentum` regime
`kappa_oracle`	True mean-reversion speed; modulates `value` strength
`num_noise_agents`	High count, low others → `noise` regime

Table 2: Parameters used to define oracle regime labels.

TODO: Define exact thresholds for the oracle heuristic. Use the value-agent crossover point from Figure 8 of the v1 report as the anchor, and determine momentum/noise boundaries empirically from baseline sweep data.

3.3 Learned Belief Module

The learned belief module is a neural network classifier:

$$\hat{b}_t = f_\phi(o_{t-k}, \ldots, o_t) \in \Delta^2,$$

trained with cross-entropy loss against oracle labels:

$$\mathcal{L}(\phi) = -\mathbb{E}\big[\log \hat{b}_t[\hat{r}]\big].$$

Training data is generated by rolling out ABIDES episodes across the regime sweep, recording $(h_t, \hat{r})$ pairs. The most informative input signals are expected to be the lagged returns (autocorrelation structure differs across regimes) and order book imbalance persistence.

The module is frozen after Phase 1, before any PPO updates begin in Phase 2.

4 Experiments

4.1 One-Day Execution Plan

The full one-at-a-time ABIDES sweep is too expensive to make the centerpiece of the current experiment cycle. The fast sweep configuration is estimated to take about two to three hours, while the default sweep is much larger because it uses more episodes and a finer one-minute timestep. Since the sweep is primarily simulator-bound rather than GPU-bound, the three available Titan GPUs are more useful for neural policy training than for accelerating ABIDES rollouts.

We therefore use the sweep only to choose and justify regimes, then spend the main compute budget on policy experiments. The targeted sweep varies only the parameters most directly tied to latent regime:

`num_value_agents`, `num_momentum_agents`, `kappa_oracle`, `fund_vol`.

This produces empirical evidence for value-like, momentum-like, and noisy market conditions without spending the full day on background-agent sensitivity analysis.

4.2 Policy Comparisons

The main experimental runs, in priority order, are:

Run 1. Baseline. $b_t = \varnothing$. Standard RL agent sees only o_t. This is $\pi_{\text{base}}(o_t)$, trained with PPO. Establishes the performance ceiling of a belief-blind policy.

Run 2. Oracle as observation. $b_t = b_t^{\text{oracle}}$ (ground truth one-hot regime label). Policy sees $[o_t, b_t^{\text{oracle}}]$. If this outperforms the baseline, regime information is provably useful and the belief-aware architecture is validated in principle.

Run 3. Oracle as reward shaping. The policy sees only o_t, but the reward is augmented with a small regime-consistency term. For example, a momentum regime can reward trend-following actions and a value regime can reward mean-reversion actions. This is less clean than the observation oracle because it changes the objective, but it tests whether oracle knowledge can guide exploration and improve learning speed.

Run 4. PPO modification. Keep the baseline observation and reward, but vary policy-training choices such as entropy regularization, learning rate, MLP width, and observation normalization. This tests whether the baseline PPO agent is under-trained before attributing gains to belief information.

Run 5. Inferred belief. $b_t = \hat{b}_t$ (output of frozen learned classifier). Policy sees $[o_t, \hat{b}_t]$. The gap between Run 2 and Run 3 quantifies how much is lost by estimating the regime rather than knowing it. This is the belief-aware experiment after the oracle experiments establish that regime information is useful.

We do not prioritize SAC for this cycle. The daily-investor action space is discrete (`BUY`, `HOLD`, `SELL`), so PPO is a natural fit and is simpler to compare against the oracle variants. If a second algorithm is needed, DQN is a lower-risk comparison because the repository already contains RLlib DQN runners, whereas SAC would require additional discrete-action support and debugging.

Metric	What it measures
Mean final P&L	Primary trading performance
Sharpe ratio	Risk-adjusted return (episode-level)
Win rate	Fraction of episodes with non-negative P&L
Max drawdown	Tail risk during episode
Belief accuracy	Classifier accuracy vs. oracle labels (Runs 2–3 only)

Table 3: Evaluation metrics across the policy and belief-aware runs.

4.3 Evaluation Metrics

4.4 Training Distribution

For policy learning (Phase 2), all three runs use a **randomized regime distribution**: ABIDES background parameters are sampled across the three regime definitions each episode. This prevents any single calibration from dominating and forces the policy to generalize across regimes. The belief module itself is *not* trained in this phase; it is already frozen and only produces $\hat{b}_t$ features. Evaluation uses held-out seeds with known regime labels.